Docker 部署 Web 服务

Apache · Tomcat · Nginx

目标

中间件也能"镜像 + 挂载"一条龙

- 共同套路: 选镜像 + `-p` 端口 + `-v` 挂载
- Apache(重点): httpd · htdocs · httpd.conf
- Tomcat: war 丢 webapps 自动部署
- Nginx: 静态站 + `proxy_pass` 反代

一、Web 服务怎么部署

> 和应用一个套路

三种耳熟能详的服务

服务	官方镜像	一句话定位
Apache	`httpd`	经典老牌 Web 服务器
Tomcat	`tomcat`	Java servlet 容器(war)
Nginx	`nginx`	高性能静态 + 反向代理

> 都是"拉镜像 + -p 映射 + 挂载定制"

换内容不改镜像: 把网站/配置挂进容器

```
docker run -d -p 8080:80 镜像
# 定制时再加 -v:
#   -v "$PWD/静态站":/站点根目录
#   -v "$PWD/配置":/配置文件
```



> 镜像只装软件, 内容随时挂在外面

二、Apache (重点)

> 传统 Web 的元老

默认跑起来, 看两处路径

```
docker run -d --name my-apache -p 8080:80 httpd:2.4
curl localhost:8080        # It works!
```

- 站点根目录: `/usr/local/apache2/htdocs`
- 配置文件: `/usr/local/apache2/conf/httpd.conf`

挂上自己的网站

```
mkdir -p my-site && echo '<h1>My Apache</h1>' > my-site/index.html

docker run -d --name my-apache -p 8080:80 \
  -v "$PWD/my-site":/usr/local/apache2/htdocs \
  httpd:2.4
curl localhost:8080        # My Apache
```



改宿主文件, 页面随时变

改配置: 挂一份 httpd.conf

```
docker cp my-apache:/usr/local/apache2/conf/httpd.conf ./httpd.conf
# 在 httpd.conf 里改 ServerName、开模块 ...

docker run -d --name my-apache -p 8080:80 \
  -v "$PWD/my-site":/usr/local/apache2/htdocs \
  -v "$PWD/httpd.conf":/usr/local/apache2/conf/httpd.conf \
  httpd:2.4
```



> 改完用 `httpd -t` 校验语法再重载, 别把服务改崩

三、Tomcat

> Java 应用的家

webapps 丢 war, 自动部署

```
docker run -d --name my-tomcat -p 8080:8080 tomcat:10
```

- Web 应用目录: `/usr/local/tomcat/webapps`
- 把 `.war` 放进去 → 自动解压发布

```
mkdir -p apps && cp sample.war apps/
docker run -d --name my-tomcat -p 8080:8080 \
  -v "$PWD/apps/sample.war":/usr/local/tomcat/webapps/sample.war \
  tomcat:10
curl localhost:8080/sample/    # 访问刚发布的应用
```

四、Nginx

> 静态王者 + 反代

静态站挂 html 根目录

```
mkdir -p web && echo '<h1>Nginx Site</h1>' > web/index.html
docker run -d --name my-nginx -p 8080:80 \
  -v "$PWD/web":/usr/share/nginx/html \
  nginx:latest
curl localhost:8080        # Nginx Site
```

- 站点根目录: `/usr/share/nginx/html`
- 配置: `/etc/nginx/conf.d/*.conf`

proxy_pass 当网关, 反向给后端容器

```
location /api/ { proxy_pass http://my-backend:5000/; }
location / { root /usr/share/nginx/html; }
```

```
docker run -d --name my-nginx -p 8080:80 \
  --network mynet \
  -v "$PWD/proxy.conf":/etc/nginx/conf.d/default.conf \
  nginx:latest
```

> Nginx 按路径分发请求 = 容器互联的实战

一表攻三

服务	站点根目录	配置	适合
Apache	htdocs	httpd.conf	传统/模块
Tomcat	webapps(war)	server.xml	Java Web
Nginx	html	conf.d	静态/代理

> Java→Tomcat; 静态/反代→Nginx; 老项目/模块→Apache

五、实战一

Apache + Nginx 双搭静态站 (作业)

> 一站点, 两个服务器各跑一遍

一份静态站, 两个服务都吃到

```
mkdir -p my-site && echo '<h1>Hello Web</h1>' > my-site/index.html

# Apache
docker run -d --name my-apache -p 8080:80 \
  -v "$PWD/my-site":/usr/local/apache2/htdocs httpd:2.4
curl localhost:8080

# Nginx
docker run -d --name my-nginx -p 8081:80 \
  -v "$PWD/my-site":/usr/share/nginx/html nginx:latest
curl localhost:8081

echo '<h1>Changed!</h1>' > my-site/index.html   # 不必重建
curl localhost:8080  # Apache 变
curl localhost:8081  # Nginx 也变
```



六、实战二

Tomcat 挂一个 web 应用

> 预习案例二

webapps 目录名 = 访问路径

```
mkdir -p myapp && echo '<h1>Hi Tomcat</h1>' > myapp/index.html

docker run -d --name my-tomcat -p 8082:8080 \
  -v "$PWD/myapp":/usr/local/tomcat/webapps/myapp tomcat:10
curl localhost:8082/myapp/    # 访问 /myapp
```



> 文件夹 myapp → URL /myapp/, 下一讲把里面的 war 换成真 Java 应用

一张表带走全部要点

方面	要点
套路	选镜像 + `-p` 端口 + `-v` 挂配置/网站
Apache	htdocs 根 · httpd.conf · `httpd -t`
Tomcat	war 丢 webapps 自动部署 · 目录名=路径
Nginx	html 根 · conf.d · proxy_pass 反代
定制	尽量挂载, 改内容不重建
选型	Java→Tomcat; 静态/代理→Nginx; 传统→Apache

网站能挂, 配置能改, 动静只在一挂之间

下一步: 部署数据库(Redis、MySQL、MongoDB) —— 有状态服务, 派上用场的是网络与持久化

谢谢大家

Q & A